STAFF REVIEW · H446 247 source models

Separate from the eight-page pupil workbook. This copy is in a public repository and is NOT access controlled. Check technical correctness before use.

Printed question mapping: Booklet Q1 = original online Q1; Booklet Q2 = original online Q2; Booklet Q3 = original online Q3

Do Now — source answers

1. Defined success criteria and unambiguous rules.

2. A requirement that must hold before an algorithm runs.

3. The pivot (under the specified partition method).

4. Stack bottom-to-top [4,9]; pop gives 2; queue [7,8,2] then dequeue 7 leaves [8,2].

I Do — source worked steps

1. Predict the method — Check the middle 5 and discard indexes 0–2 because target 7 is larger.

2. Check and explain — Next middle is index 3 containing 7, so return 3.

We Do — guided CHECK

1. Predict the method
Swap 5 and 2 to get [2,5,4].
Check and explain
Swap 5 and 4. The first full pass ends at [2,4,5].

You Do — source mark schemes

1. MARK SCHEME /15. (a) Input is sorted into the ordering the algorithm expects [1]; comparisons can then discard the half that cannot contain the target [1]. (b) Start low0 high6 mid3→7 [2]; set low4 [1]; low4 high6 mid5→11 [2]; return index5 [2]. (c) Linear O(n) [2], may check all n [1]; binary O(log n) [2], halves the sorted candidate interval [1].

2. MARK SCHEME /15. (a) Pass 1: [1,4,2,5] [3]; pass 2: [1,2,4,5] [3]; Pass 3 compares remaining adjacent values, makes no swaps, and stays [1,2,4,5] [3]. The no-swap pass confirms completion. (b) Bubble worst $O(n^2)$ [2]; merge sort worst O(n log n) [2]; merge generally uses O(n) auxiliary storage for merging while in-place bubble uses O(1) auxiliary [2].

3. MARK SCHEME /15. (a) static int SumTo(int n) [2]; if(n==0)return 0; [2]; return n+SumTo(n-1); [4] with valid braces/structure. (b) SumTo(3)=3+SumTo(2) [1], =3+2+SumTo(1) [1], =3+2+1+SumTo(0) [1], SumTo(0)=0 [1], result 6 [1]. (c) Further call-stack memory [1] and possible stack overflow on deep recursion [1].

Exit Ticket — source answer

1. False. It may recurse until stack exhaustion.